

Enterprise Java

Unit- 4

Chapter-2

Working with Session Beans

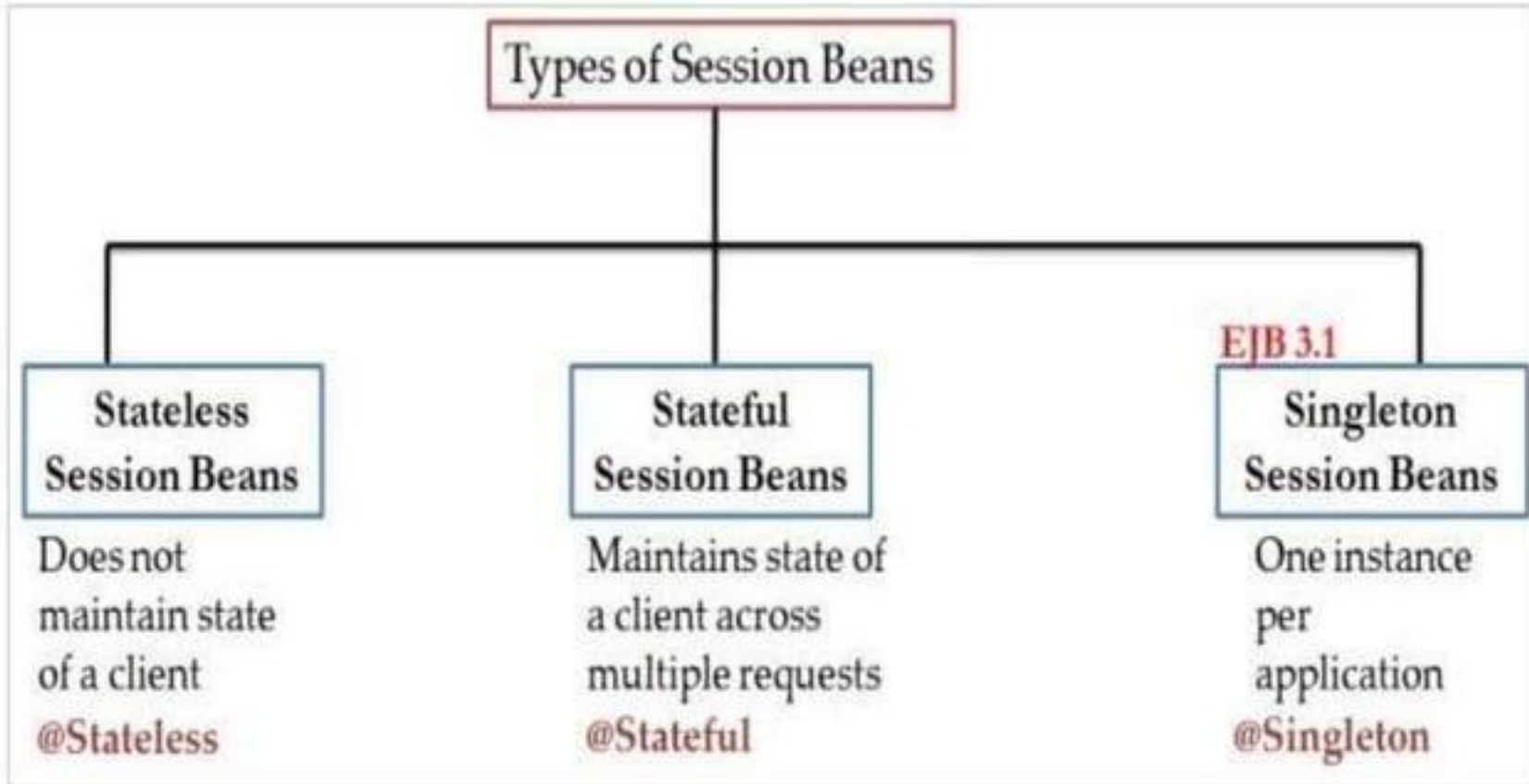
- **Session Bean**
- **Types of Session Bean**
- **Remote and Local Interface**
- **Lifecycle of Enterprise Beans**
- **Packaging Enterprise Beans**

Prof. Sandeep Vishwakarma

SESSION BEANS

- A Session Bean implements and encapsulates a business task that can be invoked programmatically by a client over local, remote, or web service client views.
- The session bean performs work (such as calculations or database access) for its client by executing business tasks inside the server.
- The application container or EJB container manages the life cycle of the session bean instances. It notifies the instances when bean action may be necessary, and it provides a full range of services to ensure that the session bean implementation is scalable and can support a large number of clients.

TYPES OF SESSION BEANS



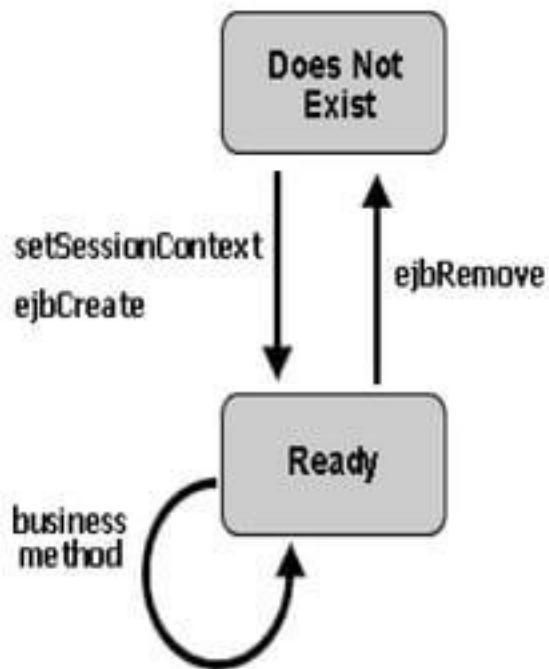
STATELESS SESSION BEAN

- Stateless Session Beans are business objects that do not have state associated with them.
- The **@Stateless** annotation is used to mark the class as Stateless Session Bean.
- Access to a single bean instance is still limited to only one client at a time and concurrent access to the bean is prohibited.
- Instance variables can be used but the contents of those **instance variables** are **not** guaranteed to be **preserved across different client method calls**.
- **Instances of Stateless Session beans** are typically **pooled** because all stateless bean instances are equivalent when they are not involved in servicing a client-invoked method.

Examples

- Sending an e-mail to customer
- Adding an item to the database
- Getting a particular list of items from the database
- Any operation which can be completed in one single shot.

STATELESS SESSION BEAN LIFE CYCLE



STATELESS SESSION BEAN LIFE CYCLE STEPS

- A session bean instance's **life starts** when a **client obtains a reference** to a stateful session bean instance **through dependency injection or JNDI lookup**.
- Stateless Session Bean is never passivated because it has only two stages.
- The EJB Container creates and maintains a pool of stateless session beans, beginning its lifecycle .
- The container performs any dependency injection and then invokes the method annotated **@PostConstruct** .
- At the end of the life cycle , the EJB Container calls the method annotated **@PreDestroy** , if it exists. The bean instance is then ready for garbage collection

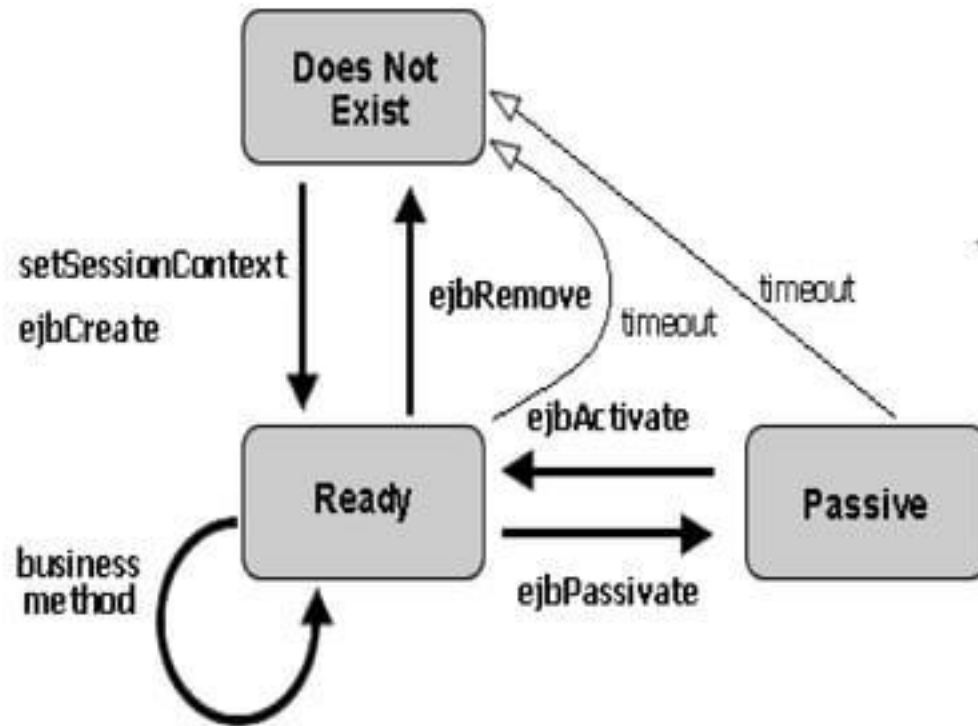
STATEFUL SESSION BEAN

- Stateful Session Beans are business objects with **@Stateful** annotation .
- Stateful Session Beans keep track of which calling client they are dealing with throughout a session.
- A session bean is not shared and access to the bean instance is strictly limited to only one client at a time.
- When the client terminates, its session bean appears to terminate and is no longer associated with the client.
- When the client invokes a method on the bean marked with **@Remove**, it signals the end of the session with the bean.
- If the client removes the bean, the session ends and the state disappears.

Examples

- Shopping cart can be implemented by Stateful session bean where the list of items added to the cart by a user is stored in an instance variable of session bean. When a new item is added to the cart all the previous items will still be maintained by the bean until the session ends.

STATEFUL SESSION BEAN LIFE CYCLE



STATEFUL SESSION BEAN LIFE CYCLE STEPS

- A session bean instance's **life starts** when a **client obtains a reference** to a stateful session bean instance **through dependency injection or JNDI lookup**.
- The container performs any dependency injection and then invokes the method annotated **@PostConstruct** ,if any. The bean is now ready to have its business methods.
- While in the ready stage, EJB Container may decide to deactivate or passivate.
- EJB Container call **@PrePassivate** then call method annotated **@PostActivate**.
- At the end of the life cycle , the client invoke a method annotated **@Remove** and EJB Container calls method annotated **@PreDestroy** if any. The bean instance is then ready for garbage collection

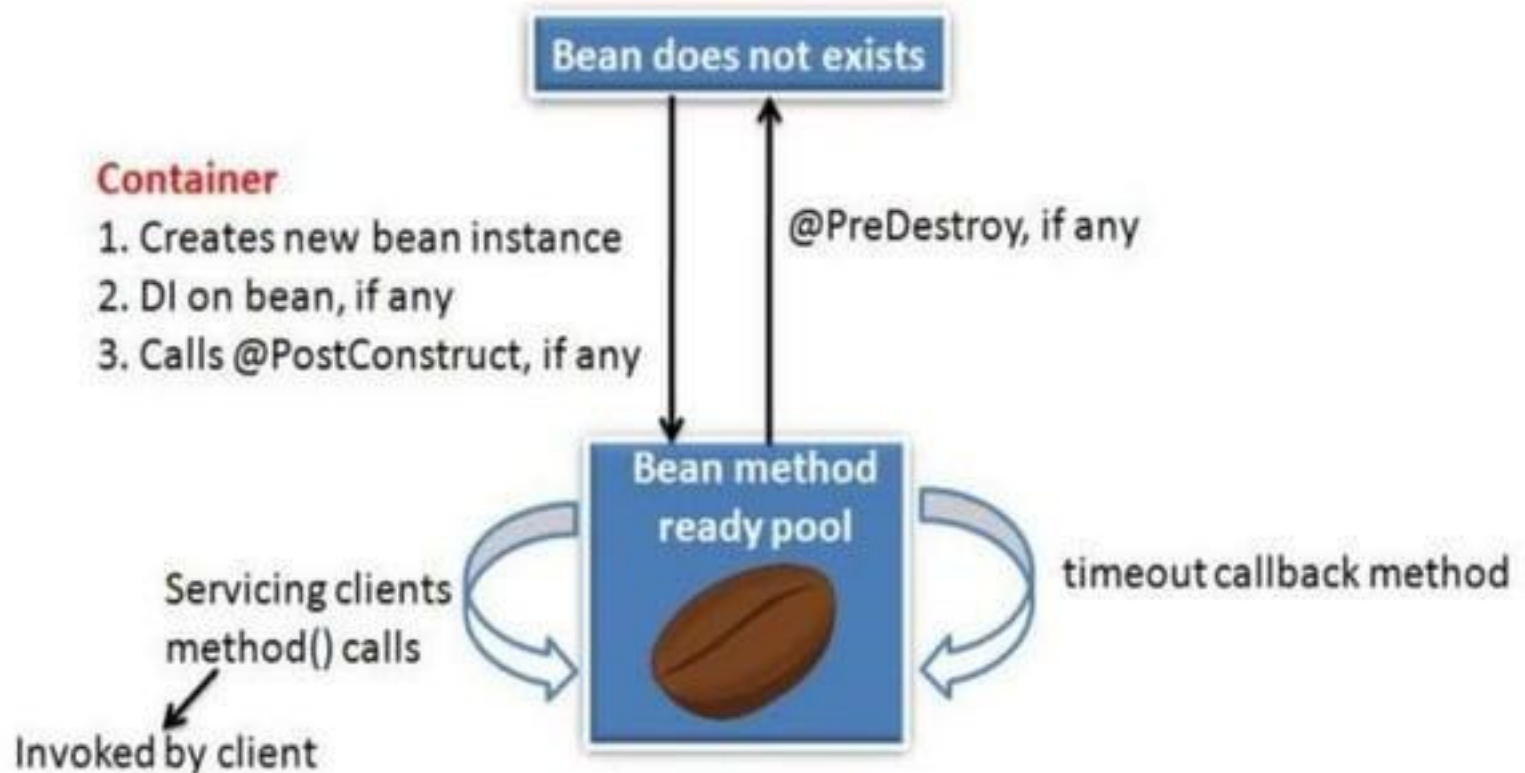
SINGLETON SESSION BEAN

- Available from **EJB 3.1**, Singleton Session Beans are business objects having a global shared state within a JVM.
- The **@Singleton** annotation is used to mark the class as Singleton Session Bean.
- They are **instantiated once per application** and exist for the lifecycle of the application.
- In cases where the container is distributed over many virtual machines, each application will have one bean instance of the Singleton for each JVM.
- It **maintains its state between client invocations** but that state is not required to survive container shutdown or crash.
- A Singleton session bean is intended to be **shared and supports concurrent access by clients**.

Examples

- Loading a global daily price list that will be the same for every user might be done with a singleton session bean, since this will prevent the application having to do the same query to a database over and over again.
- Application hit counter

SINGLETON SESSION BEAN LIFECYCLE



DI – Dependency Injection

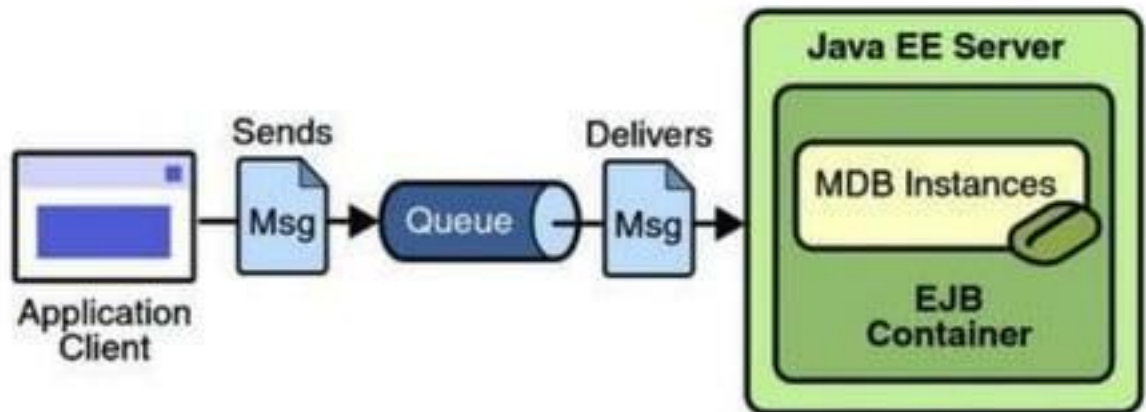
- A singleton session bean instance's **life starts when the container invokes the newInstance method** on the session bean class to create the singleton bean instance.
- Next, the **container performs any dependency injection** as specified by the metadata annotations on the bean class or by the deployment descriptor.
- If singleton is annotated with **@Startup** annotation then container performs any dependency injection and invokes **@PostConstruct** ,if any. The singleton **bean instance is now ready** to have a business method by the client .
- At the end of the life cycle , the EJB Container calls the method annotated **@PreDestroy** if exists. and EJB Container calls method anootated **@PreDestroy** if any. The singleton bean is now ready for garbage collection

What is a Message-Driven Bean?

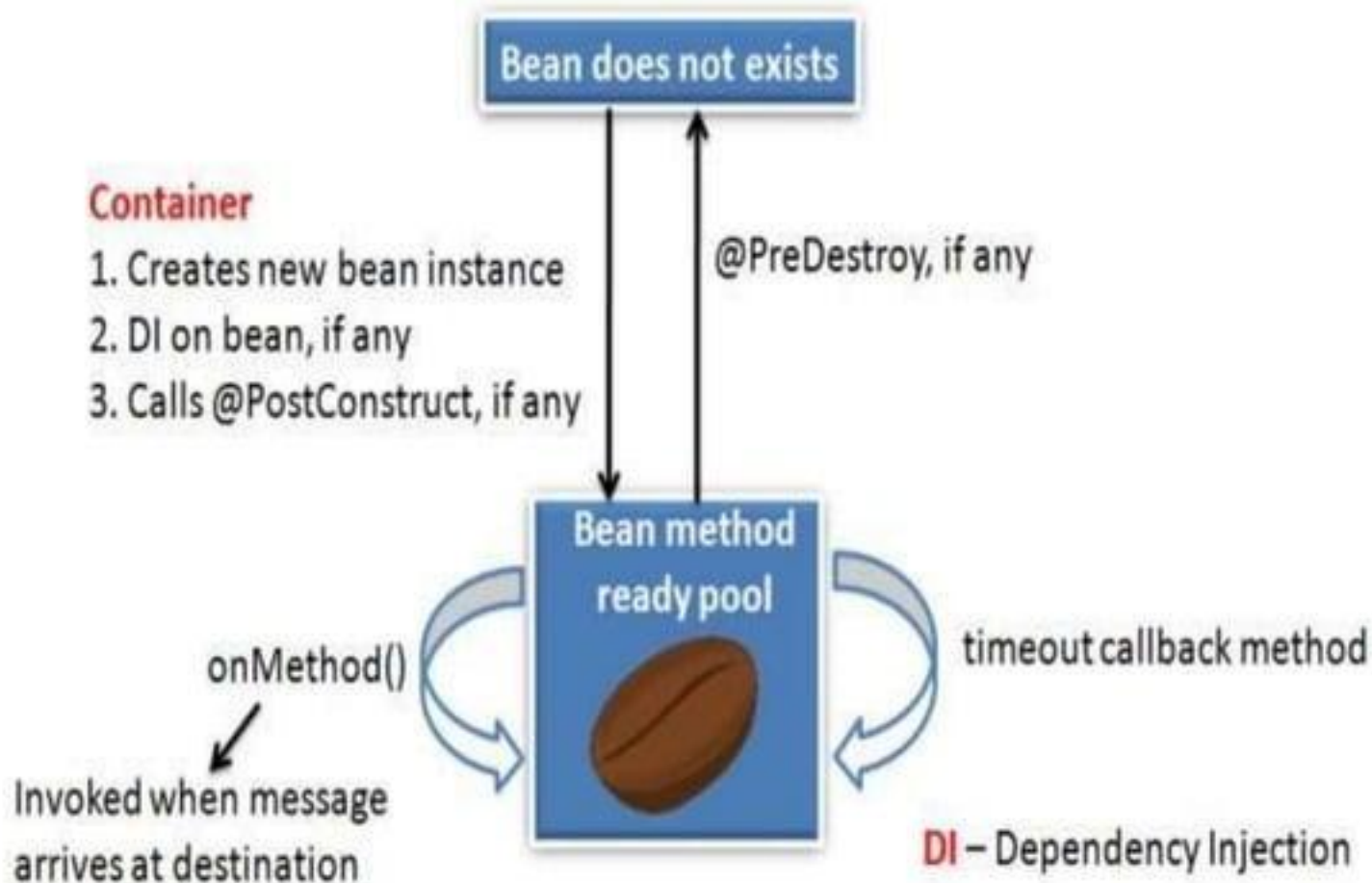
- A **message driven bean** is a type of enterprise bean, which is invoked by EJB container when it receives a message from queue or topic. Message driven bean is a stateless bean and is used to do task asynchronously.
- A **message-driven bean** is an enterprise bean that allows Java EE applications to process messages asynchronously. This type of bean normally acts as a JMS message listener, which is similar to an event listener but receives JMS messages instead of events.

Characteristics of Message-driven beans

- They execute upon receipt of a single client message.
- They are invoked asynchronously.
- They are relatively short-lived.
- They do not represent directly shared data in the database, but they can access and update this data.
- They can be transaction-aware.
- They are stateless.



Message-Driven Bean Life Cycle



Message-Driven Bean Life Cycle steps

- A message-driven bean instance's life starts when the **container invokes newInstance** on the message-driven bean class to create a new instance.
- Next, the container injects the bean's MessageDrivenContext, if applicable, and **performs** any other **dependency injection** as specified by metadata .
- The container then calls the bean's **PostConstruct lifecycle** callback methods, if any.
- The message-driven bean instance is now ready to be delivered a message sent to its associated destination or endpoint by any client.
- When the container no longer needs the instance , the container invokes the **PreDestroy lifecycle callback methods** for it, if any. This ends the life of the message-driven bean instance.